MASTER 1 – ARMAC

Computer Vision YOLO TP

YOLOv8: Detection, Fine-tuning, Tracking, and Segmentation

Group Members:

Munad E ALI
Timothée BALLANDRAS

Submitted to: **Prof Emmanuel RAMASSO**

Monday 2nd March 2026

Contents

1 Introduction **1**
1.1 Objectives . . . 1
1.2 Tools and Reproducibility . . . 1

2 Experimental Setup and Parameters **1**
2.1 Common Parameters . . . 1
2.2 Ultralytics Output Structure . . . 1

3 Part 1: YOLOv8 Theory and Image Inference **2**
3.1 Theory: Loss, Model Naming, and Tasks . . . 2
3.1.1 Composite Loss Components . . . 2
3.1.2 Why CIoU/DIoU/GIoU Instead of IoU Alone . . . 2
3.1.3 Model Naming: `yolov8n.pt` . . . 2
3.1.4 `best.pt` vs `last.pt` . . . 2
3.1.5 YOLO Tasks . . . 3
3.2 Inference on Provided Images . . . 3
3.2.1 Interpreting the Results Object . . . 3
3.3 Hyperparameter Study: `conf` and `iou` . . . 4
3.3.1 Effect of Confidence Threshold (`conf`) . . . 4
3.3.2 Effect of IoU Threshold (`iou`) and NMS . . . 5
3.3.3 Why the Main Object Might Not Be Detected . . . 6
3.4 Class Filtering: People-only on `bravo.jpeg` . . . 6

4 Part 2: Vehicle Dataset Evaluation and Fine-tuning **6**
4.1 Dataset Format (YOLO) . . . 6
4.2 Baseline Evaluation (Pretrained YOLOv8n) . . . 7
4.2.1 Baseline Metrics . . . 7
4.3 Fine-tuning (Transfer Learning) . . . 7
4.3.1 Run Folder Outputs . . . 7
4.4 Evaluation After Fine-tuning and Comparison . . . 9
4.4.1 Fine-tuned Metrics . . . 9
4.4.2 Absolute Improvements (Filled) . . . 9
4.4.3 Qualitative Comparison (Side-by-side) . . . 9
4.5 Discussion . . . 10
4.6 Theory: Training from Scratch vs Fine-tuning . . . 10
4.6.1 Training from Scratch . . . 10
4.6.2 Fine-tuning Procedure . . . 10

5 Part 3: Video Detection, FPS, and Tracking **10**
5.1 Video Detection + Timing + Tracking (Efficient) . . . 10
5.2 Video Metadata . . . 11
5.3 Stride Parameter and Practical Choices . . . 11
5.4 Theoretical Discussion (Detection & Tracking) . . . 11
5.4.1 Confidence Threshold (`conf`) . . . 11
5.4.2 IoU Threshold (`iou`) in Non-Maximum Suppression (NMS) . . . 11
5.4.3 Why Tracking Is Slower Than Detection . . . 11
5.4.4 Real-Time Constraint . . . 12
5.5 Detection Output and FPS . . . 12
5.6 Visual Results (Detection Frames) . . . 12
5.7 Tracking with ByteTrack . . . 12

5.8 Visual Results (Tracking Frames) . . . 12

6 Part 4: Drone Dataset Case Study (VisDrone) 13
6.1 Dataset and Main Challenges . . . 13
6.2 Training Setup (Hyperparameters and Configuration) . . . 13
6.3 Quantitative Results . . . 13
6.4 Per-Class Analysis . . . 14

7 Part 5: Segmentation and Backbone Benchmarking 15
7.1 Segmentation vs Detection . . . 15
7.2 When Segmentation is Necessary . . . 15
7.3 Backbone / Model Size Benchmark (YOLOv8n vs s vs m) . . . 16
7.4 Discussion: Speed vs Performance . . . 16

8 Conclusion 16

A Appendix A: Key Code Snippets (Curated) and Explanations 16
A.1 A.1 Environment Setup and Imports . . . 17
A.2 A.2 Basic Inference on an Image . . . 17
A.3 A.3 Confidence Sweep (`conf`) . . . 17
A.4 A.4 IoU Sweep (`iou`) and Grid Saving . . . 18
A.5 A.5 Class Filtering (People-only) . . . 18
A.6 A.6 Dataset YAML (Vehicles) and Validation . . . 18
A.7 A.7 Fine-tuning on Vehicles (Transfer Learning) . . . 19
A.8 A.8 Validate Fine-tuned Model . . . 19
A.9 A.9 Side-by-side Predictions (Baseline vs Fine-tuned) . . . 19
A.10 A.10 Video Detection + FPS Measurement . . . 19
A.11 A.11 Tracking with ByteTrack . . . 20
A.12 A.12 VisDrone Training . . . 20
A.13 A.13 Detection vs Segmentation on Video Frames . . . 20
A.14 A.14 Benchmark YOLOv8n/s/m on Video . . . 21

B Appendix B: Extra Figures 21

List of Figures

1 Example directory structure illustrating typical Ultralytics outputs. . . . 2
2 Input images used for Part 1 inference. . . . 3
3 Example printout of the results object for interpretation. . . . 4
4 Detections summarized in a table (class, confidence, coordinates). . . . 4
5 Detection count as a function of confidence threshold. . . . 5
6 Grid of predictions for different `(conf, iou)` settings demonstrating NMS effects. 5
7 Class filtering on `bravo.jpeg`: keeping only persons. . . . 6
8 Vehicle dataset structure (train/valid/test). . . . 6
9 Example YOLO label file content (normalized coordinates). . . . 6
10 Dataset YAML example (`vehicles.yaml`): `train/val/test` paths, `nc`, `names`. . 7
11 Example listing of files generated in the training run directory. . . . 7
12 Training curves (`results.png`): losses and metrics over epochs. . . . 8
13 Confusion matrix on the validation set after fine-tuning. . . . 8
14 Overall and class-based performance of the finetuned model . . . 9
15 Vehicles: comparison on test image #1 (baseline vs fine-tuned). . . . 9
16 Vehicles: comparison on test image #2 (baseline vs fine-tuned). . . . 10

17 Detection results on selected frames from the annotated output video. 12
18 ByteTrack tracking output: bounding boxes + class/confidence + track IDs on selected frames. 13
19 VisDrone training curves (`results.png`). 14
20 VisDrone confusion matrix (validation). 14
21 Detection vs segmentation on the same frame example 15
22 Vehicles: Precision-Recall curve 21
23 Vehicles: F1 curve vs confidence threshold 22

List of Tables

1 Detection count for each confidence threshold 5
2 Baseline vs fine-tuned performance on vehicles validation set (filled). 9
3 Test video metadata 11
4 Video performance summary (achieved FPS). 12
5 VisDrone training hyperparameters (from implementation). 13
6 VisDrone validation metrics (filled). 13
7 Benchmark results (filled from your code). 16

1 Introduction

This report presents a complete study of YOLOv8 across five parts: image inference and threshold analysis, domain adaptation via fine-tuning on a vehicle dataset, video detection with FPS evaluation, multi-object tracking with ByteTrack, a drone dataset case study (VisDrone), segmentation vs detection comparisons, and backbone/model-size benchmarking. The objective is both to demonstrate correct implementation and to provide evidence-based analysis (metrics + figures).

1.1 Objectives

- **Part 1:** Run YOLO inference on provided images, interpret outputs, study `conf` and `iou`, and filter detections by class.
- **Part 2:** Evaluate a pretrained model on a vehicle dataset, fine-tune it, and compare results quantitatively and visually.
- **Part 3:** Perform video object detection, compute achieved FPS, and run tracking with object IDs.
- **Part 4:** Train on a drone dataset (VisDrone) and analyze global and per-class performance with qualitative examples.
- **Part 5:** Compare detection vs segmentation outputs and benchmark YOLOv8n/s/m speed vs model size.

1.2 Tools and Reproducibility

Experiments were conducted with Ultralytics YOLOv8 (Python), with OpenCV/Matplotlib/-Pandas for processing and visualization. Ultralytics stores outputs under `runs/` (predictions, training curves, best/last checkpoints, confusion matrices, PR curves, etc.). All results shown in this report can be regenerated by running the provided scripts/notebooks.

2 Experimental Setup and Parameters

2.1 Common Parameters

The main parameters used across experiments are:

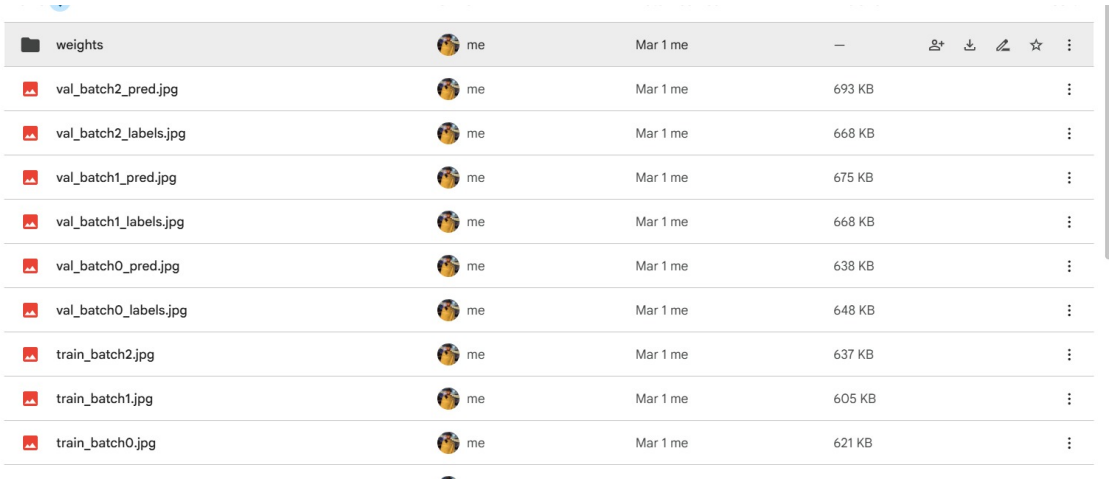
- `imgsz`: input image size (common default: 640; sometimes 512 in drone runs).
- `conf`: confidence threshold to keep detections.
- `iou`: IoU threshold for NMS suppression of duplicate boxes.
- `device`: CPU/GPU execution selection.
- `vid_stride`: frame subsampling to trade speed vs smoothness/accuracy.

2.2 Ultralytics Output Structure

Ultralytics creates run folders under `runs/`, typically containing:

- `weights/best.pt`: best validation checkpoint
- `weights/last.pt`: final epoch checkpoint

- `results.png`: training curves (loss + metrics)
- confusion matrices and PR/F1 curves for validation



weights	me	Mar 1 me	–	
val_batch2_pred.jpg	me	Mar 1 me	693 KB	
val_batch2_labels.jpg	me	Mar 1 me	668 KB	
val_batch1_pred.jpg	me	Mar 1 me	675 KB	
val_batch1_labels.jpg	me	Mar 1 me	668 KB	
val_batch0_pred.jpg	me	Mar 1 me	638 KB	
val_batch0_labels.jpg	me	Mar 1 me	648 KB	
train_batch2.jpg	me	Mar 1 me	637 KB	
train_batch1.jpg	me	Mar 1 me	605 KB	
train_batch0.jpg	me	Mar 1 me	621 KB	

Figure 1: Example directory structure illustrating typical Ultralytics outputs.

3 Part 1: YOLOv8 Theory and Image Inference

3.1 Theory: Loss, Model Naming, and Tasks

YOLOv8 is a one-stage detector trained with a composite objective combining localization and classification, with additional terms to improve regression quality. Practical performance depends on the dataset distribution and the selected inference thresholds.

3.1.1 Composite Loss Components

- **Box regression loss:** penalizes inaccurate box geometry (location and size).
- **Classification loss:** penalizes incorrect class predictions.
- **DFL (Distribution Focal Loss):** improves bounding-box precision by modeling regression as a distribution.

3.1.2 Why CIoU/DIoU/GIoU Instead of IoU Alone

Plain IoU provides weak learning signals when boxes do not overlap. Improved IoU losses introduce terms such as center distance, enclosing box size, or aspect ratio, providing more stable gradients and better convergence.

3.1.3 Model Naming: `yolov8n.pt`

The suffix indicates the model scale (nano, small, medium, large, extra-large). Smaller models run faster but may reduce accuracy.

3.1.4 `best.pt` vs `last.pt`

`best.pt` is selected by validation performance (commonly best mAP), while `last.pt` is the checkpoint at the final epoch. Deployment typically uses `best.pt`.

3.1.5 YOLO Tasks

YOLOv8 supports detection, classification, segmentation, pose estimation, and tracking. This report focuses on detection, tracking, and segmentation.

3.2 Inference on Provided Images

We applied YOLOv8 detection to provided images: `zidane.jpg`, `bus.jpg`, and `bravo.jpeg`. The output includes predicted bounding boxes, class IDs, names, and confidence scores.

(a) Original: Zidane

(b) Original: Bus

(c) Original: Bravo

Figure 2: Input images used for Part 1 inference.

3.2.1 Interpreting the Results Object

The inference result object exposes:

- `r.boxes`: bounding boxes and scores
- `r.names`: class mapping (ID → name)
- `r.probs`: probabilities (task-dependent)

Figure 3: Example printout of the results object for interpretation.

	cls_id	conf	x1	y1	x2	y2	cls_name
0	0	0.805568	123.452026	197.184021	1110.561768	710.509766	person
1	0	0.793672	746.568726	40.801331	1142.080566	712.365967	person
2	27	0.370219	436.794067	437.129547	523.534546	713.736450	tie

Figure 4: Detections summarized in a table (class, confidence, coordinates).

3.3 Hyperparameter Study: `conf` and `iou`

3.3.1 Effect of Confidence Threshold (`conf`)

Increasing `conf` usually reduces false positives but can remove correct low-confidence detections (lower recall). We performed a sweep over confidence values and recorded the number of detections.

	image	conf	iou	detections	classes_detected
0	zidane.jpg	0.05	0.7	20	person, tie, tennis racket, baseball bat, cell...
1	zidane.jpg	0.10	0.7	9	person, tie
2	zidane.jpg	0.15	0.7	6	person, tie
3	zidane.jpg	0.25	0.7	3	person, tie
4	zidane.jpg	0.40	0.7	2	person
5	zidane.jpg	0.60	0.7	2	person

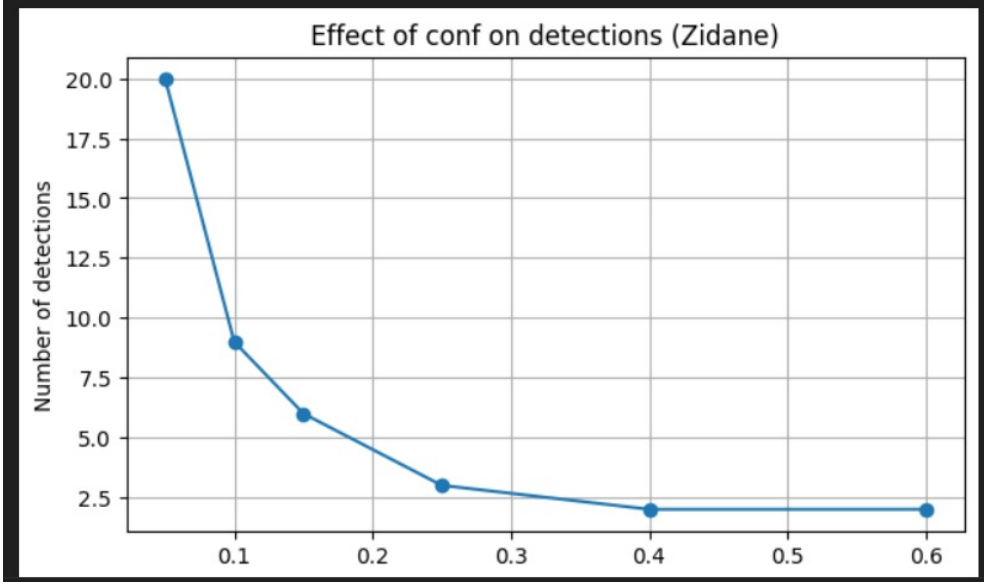


Figure 5: Detection count as a function of confidence threshold.

Table 1: Detection count for each confidence threshold

conf	# detections
0.10	9
0.25	3
0.40	2
0.60	2

3.3.2 Effect of IoU Threshold (`iou`) and NMS

Non-Maximum Suppression removes duplicate detections with high overlap. Lower `iou` thresholds suppress aggressively and may remove true nearby objects; higher thresholds may keep duplicates.

	conf	iou	detections	classes_detected	output_folder	output_image
0	0.10	0.7	9	person, tie	runs/detect/zidane_conf0.1_iou0.7	zidane.jpg
1	0.25	0.3	2	person, tie	runs/detect/zidane_conf0.25_iou0.3	zidane.jpg
2	0.25	0.7	3	person, tie	runs/detect/zidane_conf0.25_iou0.7	zidane.jpg
3	0.25	0.9	4	person, tie	runs/detect/zidane_conf0.25_iou0.9	zidane.jpg
4	0.50	0.7	2	person	runs/detect/zidane_conf0.5_iou0.7	zidane.jpg

Figure 6: Grid of predictions for different `(conf, iou)` settings demonstrating NMS effects.

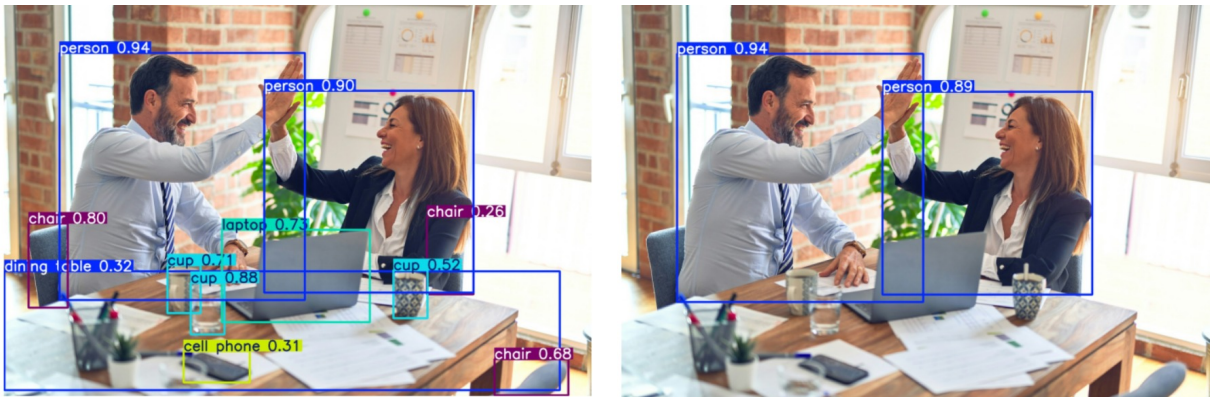
3.3.3 Why the Main Object Might Not Be Detected

Typical causes:

- **Domain mismatch:** pretrained dataset differs from the target images.
- **Difficult image conditions:** occlusion, motion blur, small objects, low contrast.
- **Thresholding:** a too-high `conf` or too-strict NMS may remove correct detections.

3.4 Class Filtering: People-only on `bravo.jpeg`

We filtered detections to keep only the `person` class.



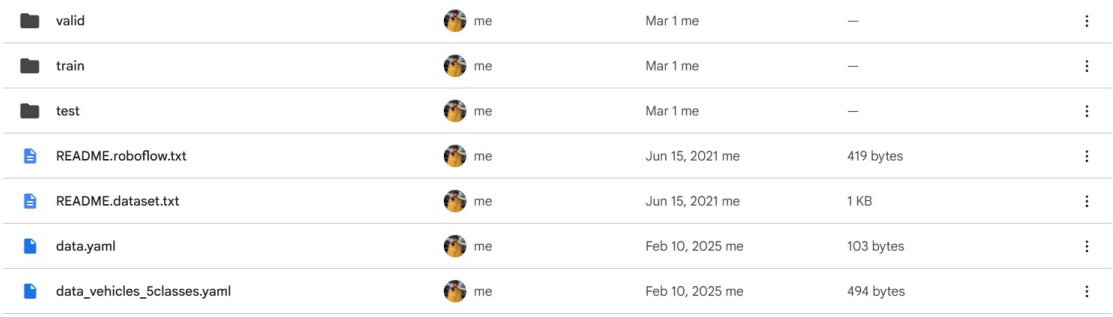
(a) Default detection (b) People-only

Figure 7: Class filtering on `bravo.jpeg`: keeping only persons.

4 Part 2: Vehicle Dataset Evaluation and Fine-tuning

4.1 Dataset Format (YOLO)

The vehicle dataset follows YOLO conventions: one `.txt` label file per image, with each line storing: `class_id x_center y_center width height` normalized in $[0, 1]$. A dataset YAML file defines paths and class names.



valid	me	Mar 1 me	—	⋮
train	me	Mar 1 me	—	⋮
test	me	Mar 1 me	—	⋮
README.roboflow.txt	me	Jun 15, 2021 me	419 bytes	⋮
README.dataset.txt	me	Jun 15, 2021 me	1 KB	⋮
data.yaml	me	Feb 10, 2025 me	103 bytes	⋮
data_vehicles_5classes.yaml	me	Feb 10, 2025 me	494 bytes	⋮

Figure 8: Vehicle dataset structure (train/valid/test).

```
2 0.1346153846153846 0.8653846153846154 0.2692307692307692 0.2668269230769231
2 0.3112980769230769 0.5384615384615384 0.18389423076923078 0.1466346153846154
```

Figure 9: Example YOLO label file content (normalized coordinates).

```yaml
train: train/images
val: valid/images

nc: 5
names: ['Ambulance', 'Bus', 'Car', 'Motorcycle', 'Truck']
```

Figure 10: Dataset YAML example (`vehicles.yaml`): `train/val/test` paths, `nc`, `names`.

4.2 Baseline Evaluation (Pretrained YOLOv8n)

We evaluated the pretrained COCO model (`yolov8n.pt`) on the vehicle dataset validation set to measure domain shift.

4.2.1 Baseline Metrics

- Precision (P): **0.4218**
- Recall (R): **0.2571**
- mAP@0.5: **0.2407**
- mAP@0.5:0.95: **0.1724**

4.3 Fine-tuning (Transfer Learning)

We fine-tuned YOLOv8n on the vehicle dataset (transfer learning). GPU usage is important because training requires many forward/backward passes.

4.3.1 Run Folder Outputs

Typical Ultralytics outputs include: `weights/best.pt`, `weights/last.pt`, `results.png`, `confusion_matrix.pn` and PR/F1 curves.

weights	me	Mar 1 me	—	⋮
val_batch2_pred.jpg	me	Mar 1 me	693 KB	⋮
val_batch2_labels.jpg	me	Mar 1 me	668 KB	⋮
val_batch1_pred.jpg	me	Mar 1 me	675 KB	⋮
val_batch1_labels.jpg	me	Mar 1 me	668 KB	⋮
val_batch0_pred.jpg	me	Mar 1 me	638 KB	⋮
val_batch0_labels.jpg	me	Mar 1 me	648 KB	⋮
train_batch2.jpg	me	Mar 1 me	637 KB	⋮
train_batch1.jpg	me	Mar 1 me	605 KB	⋮
train_batch0.jpg	me	Mar 1 me	621 KB	⋮
results.png	me	Mar 1 me	319 KB	⋮

Figure 11: Example listing of files generated in the training run directory.

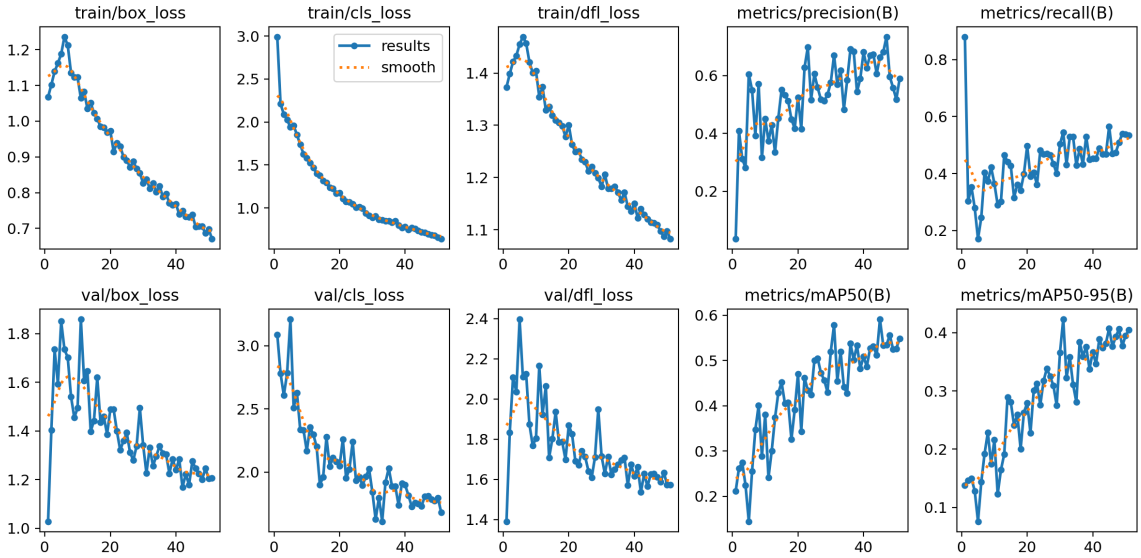


Figure 12: Training curves (`results.png`): losses and metrics over epochs.

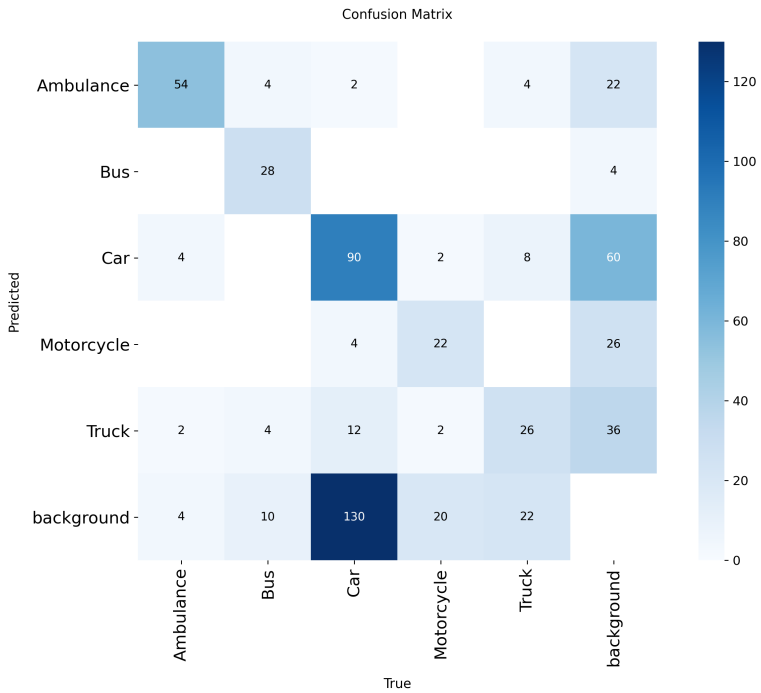


Figure 13: Confusion matrix on the validation set after fine-tuning.

4.4 Evaluation After Fine-tuning and Comparison

4.4.1 Fine-tuned Metrics

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):
all	250	454	0.671	0.544	0.578	0.425
Ambulance	50	64	0.762	0.844	0.887	0.729
Bus	30	46	0.875	0.609	0.658	0.51
Car	90	238	0.646	0.323	0.427	0.301
Motorcycle	42	46	0.597	0.478	0.48	0.287
Truck	38	60	0.476	0.467	0.44	0.296

Figure 14: Overall and class-based performance of the finetuned model

Table 2: Baseline vs fine-tuned performance on vehicles validation set (filled).

Model	**Precision**	**Recall**	**mAP@50**	**mAP@50–95**
Pretrained YOLOv8n (`yolov8n.pt`)	0.4218	0.2571	0.2407	0.1724
Fine-tuned YOLOv8n (`best.pt`)	0.6712	0.5440	0.5783	0.4247

4.4.2 Absolute Improvements (Filled)

- Precision: 0.4218 → 0.6712 (**+0.2494**)
- Recall: 0.2571 → 0.5440 (**+0.2869**)
- mAP@0.5: 0.2407 → 0.5783 (**+0.3376**)
- mAP@0.5:0.95: 0.1724 → 0.4247 (**+0.2523**)

4.4.3 Qualitative Comparison (Side-by-side)

(a) Baseline (b) Fine-tuned

Figure 15: Vehicles: comparison on test image #1 (baseline vs fine-tuned).

(a) Baseline (b) Fine-tuned

Figure 16: Vehicles: comparison on test image #2 (baseline vs fine-tuned).

4.5 Discussion

The baseline metrics are relatively low due to **domain shift** (COCO vs the vehicle dataset distribution). Fine-tuning significantly improves precision, recall, and mAP, indicating that features and classifier layers adapt to the new domain. Remaining errors typically arise from occlusions, unusual viewpoints, small objects, and class ambiguity.

4.6 Theory: Training from Scratch vs Fine-tuning

4.6.1 Training from Scratch

Training from scratch requires a large labeled dataset, extensive compute, careful hyperparameter tuning, and longer convergence time.

4.6.2 Fine-tuning Procedure

Fine-tuning starts from pretrained weights, trains on the new dataset, validates each epoch, and selects `best.pt` based on validation performance.

5 Part 3: Video Detection, FPS, and Tracking

5.1 Video Detection + Timing + Tracking (Efficient)

This part runs inference on a video using the trained YOLO model (`best.pt`) stored on Google Drive. To prevent runtime crashes and reduce RAM usage, we used:

- `stream=True` to process frames in a streaming manner (low RAM),
- `vid_stride = N` to process only every N-th frame (speed-up),
- saving the output annotated videos to Google Drive for inspection.

Input test video (Google Drive): Open link

5.2 Video Metadata

Instead of embedding a screenshot, the metadata extracted from the video is summarized below.

Table 3: Test video metadata

Property	**Value**
File name	`test_video.mp4`
Resolution	1920×1080
FPS (source video)	30.00
Total frames	607
Duration (seconds)	20.2

5.3 Stride Parameter and Practical Choices

`vid_stride` controls frame subsampling: with `vid_stride`=N, the model processes every N-th frame and skips the rest. This increases speed but can reduce temporal smoothness and may miss short events.

Detection stride selection: We initially tested `vid_stride`=10 and `vid_stride`=5, but the output was too slow for detailed analysis. Therefore, we reduced to `vid_stride`=2 to obtain a denser annotated video that is easier to analyze visually.

Tracking stride selection: For tracking, we used `vid_stride`=5 as a compromise: tracking is inherently more expensive than detection because it performs temporal association (ID assignment) across frames.

5.4 Theoretical Discussion (Detection & Tracking)

5.4.1 Confidence Threshold (`conf`)

The confidence score estimates how likely a predicted box corresponds to a real object of a class.

- Higher `conf`: fewer false positives but more missed detections (lower recall).
- Lower `conf`: higher recall but potentially more false positives.

Thus, `conf` controls the precision–recall trade-off.

5.4.2 IoU Threshold (`iou`) in Non-Maximum Suppression (NMS)

Intersection over Union (IoU) measures overlap between two boxes:

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union}.$$

During NMS, if the IoU between two predicted boxes exceeds `iou`, the lower-confidence box is suppressed.

- Lower `iou`: more aggressive suppression (fewer duplicates).
- Higher `iou`: keeps more overlapping boxes (risk of duplicates).

5.4.3 Why Tracking Is Slower Than Detection

Detection processes each frame independently. Tracking adds extra computation: data association between frames, motion consistency, and ID management (ByteTrack), which reduces FPS.

5.4.4 Real-Time Constraint

A common real-time reference is ≈ 25 FPS. If inference FPS < 25, the system cannot keep up with live playback. In our measurements, detection is near real-time, while tracking is significantly below real-time.

5.5 Detection Output and FPS

Detected video (Google Drive): Open link

From the implementation and measurements, the achieved detection performance was:

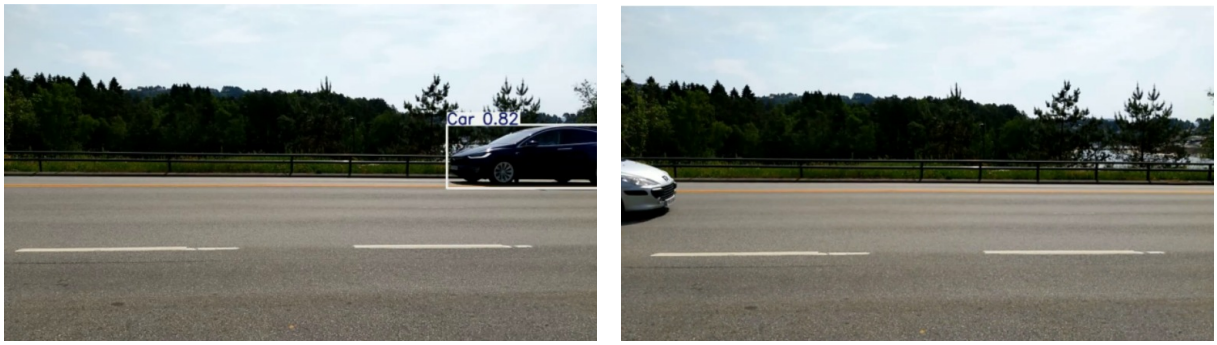
- **Detection FPS: 20.27 FPS**

Table 4: Video performance summary (achieved FPS).

Mode	Achieved FPS
Detection	20.27
Tracking (ByteTrack)	12.66

5.6 Visual Results (Detection Frames)

Only two representative frames were kept for clarity .



(a) Frame 0 (b) Frame 33

Figure 17: Detection results on selected frames from the annotated output video.

5.7 Tracking with ByteTrack

In tracking output, each detection is augmented with a persistent **Track ID**. Track IDs may switch or disappear under: occlusion, objects exiting/re-entering the field of view, intermittent detections, or aggressive frame skipping (large `vid_stride`).
Tracked video (Google Drive): Open link

The achieved tracking performance was: Tracking FPS: **12.66 FPS**

5.8 Visual Results (Tracking Frames)

Only two representative frames were kept for clarity (as available in our extracted images).

(a) Frame 30

(b) Frame 99

Figure 18: ByteTrack tracking output: bounding boxes + class/confidence + track IDs on selected frames.

6 Part 4: Drone Dataset Case Study (VisDrone)

6.1 Dataset and Main Challenges

VisDrone is challenging due to dense scenes, strong scale variation, small objects, and occlusions typical of drone viewpoints. This makes it a good stress test for small models and short training runs.

6.2 Training Setup (Hyperparameters and Configuration)

The VisDrone training was executed using Ultralytics YOLOv8 with the following configuration (as implemented in the notebook):

Table 5: VisDrone training hyperparameters (from implementation).

Hyperparameter	**Value**
Model	`yolov8n.pt`
Data YAML	`VisDrone.yaml`
Epochs	10
imgsz	512
Batch	16
Fraction	1.0
Patience	5
Device	0 (CUDA) / CPU fallback

6.3 Quantitative Results

Overall validation performance:

Table 6: VisDrone validation metrics (filled).

Precision	**Recall**	**mAP@50**	**mAP@50–95**
0.33	0.227	0.211	0.117

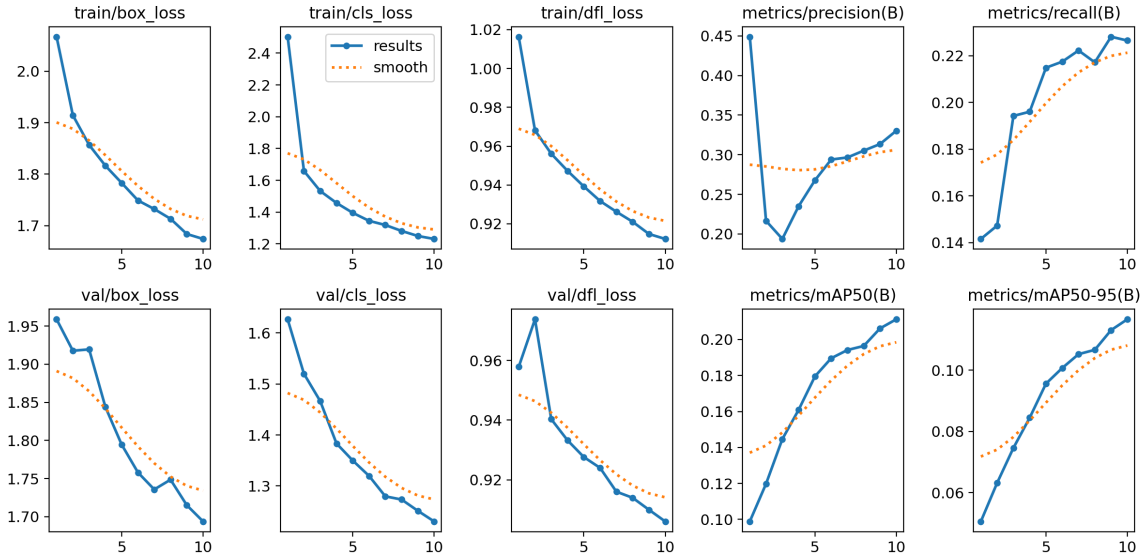


Figure 19: VisDrone training curves (`results.png`).

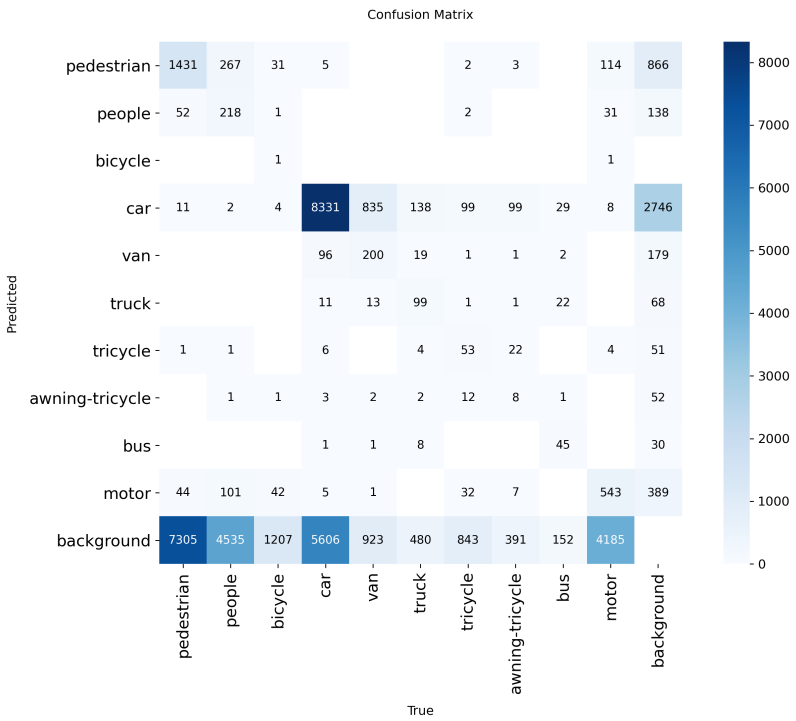


Figure 20: VisDrone confusion matrix (validation).

6.4 Per-Class Analysis

The strongest class was **car**:

- Precision: 0.468
- Recall: 0.676
- mAP@50: 0.63
- mAP@50–95: 0.395

Moderate classes:

- bus (mAP@50: 0.291), van (0.222), truck (0.194)

Weak classes (mostly small and/or occluded):

- bicycle (0.0279), awning-tricycle (0.0622), tricycle (0.115), people (0.167)

This supports the conclusion that **small-object detection is the dominant difficulty** in drone datasets under limited training and resolution.

7 Part 5: Segmentation and Backbone Benchmarking

7.1 Segmentation vs Detection

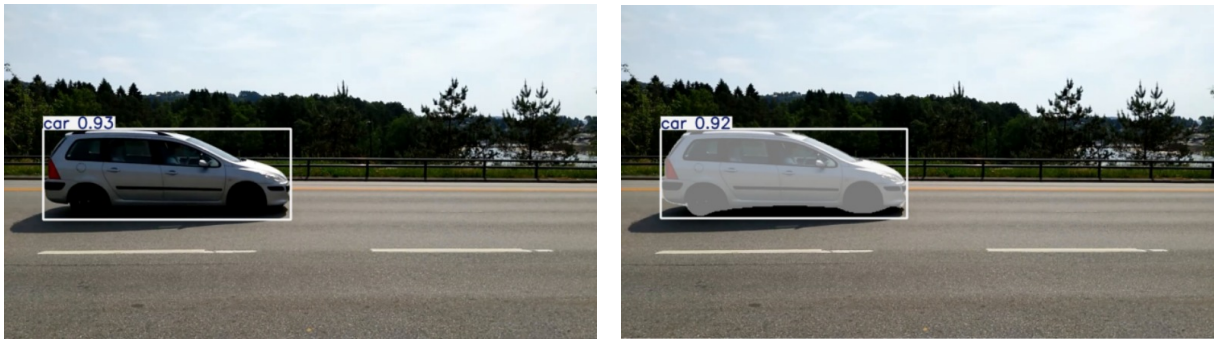
Detection outputs bounding boxes and class scores. Segmentation adds pixel-level masks, typically increasing compute and memory cost (dense mask prediction + post-processing).

Unlike detection, which predicts 4 box coordinates per object, segmentation predicts a dense mask (often at feature-map resolution) for every detected object. This requires:

- Additional mask head layers in the network,
- Per-pixel classification,
- Upsampling operations,
- Extra post-processing to project masks back to image space.

Therefore, segmentation increases:

- GPU memory usage,
- Number of floating point operations (FLOPs),
- Inference latency (lower FPS).



(a) Detection (boxes) (b) Segmentation (masks)

Figure 21: Detection vs segmentation on the same frame example

7.2 When Segmentation is Necessary

Segmentation is required when precise object boundaries matter, e.g., medical imaging (lesion boundaries), robotic manipulation (grasping), autonomous driving (drivable area estimation), industrial inspection (defect boundaries), and scene parsing.

Bounding boxes are sufficient when only object presence and approximate location are needed. Segmentation becomes indispensable when shape, area, or contour information is required.

7.3 Backbone / Model Size Benchmark (YOLOv8n vs s vs m)

We benchmarked multiple YOLOv8 model sizes under comparable video settings and recorded model sizes and speed.

Model variants (n, s, m) differ in:

- Number of parameters,
- Depth and width of convolutional layers,
- Feature representation capacity,
- Computational cost (FLOPs).

Larger models extract richer hierarchical features but require more memory and computation.

Table 7: Benchmark results (filled from your code).

Model	**Params**	**Size**	**FPS**	**Total Dets**	**Avg Dets/Frame**
YOLOv8n	3.16M	6.2 MB	25.25	66	0.55
YOLOv8s	11.17M	21.5 MB	21.13	59	0.49
YOLOv8m	25.90M	49.7 MB	19.85	67	0.55

7.4 Discussion: Speed vs Performance

The benchmark clearly illustrates the classical deep learning trade-off:

- YOLOv8n (Nano) achieves the highest FPS and lowest memory usage.
- YOLOv8s provides a balance between speed and representation capacity.
- YOLOv8m increases model capacity but reduces FPS.

The raw number of detections is not equivalent to accuracy. Proper comparison requires evaluation metrics such as mAP, precision, and recall.

Thus, model selection depends on deployment constraints:

- Real-time embedded systems → YOLOv8n
- Balanced performance → YOLOv8s
- Higher accuracy with less strict speed constraints → YOLOv8m

8 Conclusion

This work validated correct end-to-end use of YOLOv8 for image inference and interpretation, threshold sensitivity analysis, domain adaptation through fine-tuning on a vehicles dataset, video detection with FPS measurement, tracking with ByteTrack IDs, and a drone detection case study on VisDrone. Finally, detection vs segmentation and model-size benchmarking highlighted deployment trade-offs between accuracy, output richness, and inference speed.

A Appendix A: Key Code Snippets (Curated) and Explanations

This appendix groups essential code blocks used to reproduce the results presented in the report.

A.1 A.1 Environment Setup and Imports

```
import os, time, random
import numpy as np
import pandas as pd
import cv2
import matplotlib.pyplot as plt

from ultralytics import YOLO

import torch
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

Listing 1: Core imports and environment checks (GPU/CUDA).

A.2 A.2 Basic Inference on an Image

```
model = YOLO("yolov8n.pt")
r = model("zidane.jpg")[0]

# Available fields
print("names:", r.names)
print("boxes:", r.boxes)

# Access arrays
xyxy = r.boxes.xyxy.cpu().numpy()
cls  = r.boxes.cls.cpu().numpy().astype(int)
conf = r.boxes.conf.cpu().numpy()

# Convert to a readable table
df = pd.DataFrame({
    "class_id": cls,
    "class_name": [r.names[i] for i in cls],
    "conf": conf,
    "x1": xyxy[:,0], "y1": xyxy[:,1], "x2": xyxy[:,2], "y2": xyxy[:,3]
})
print(df.head())
```

Listing 2: Run YOLO inference on one image and inspect outputs.

A.3 A.3 Confidence Sweep (conf)

```
conf_values = [0.10, 0.25, 0.50, 0.75]
counts = []

for c in conf_values:
    r = model.predict("zidane.jpg", conf=c, save=False)[0]
    counts.append(len(r.boxes))

df_conf = pd.DataFrame({"conf": conf_values, "detections": counts})
print(df_conf)

# Plot
plt.figure()
```

```
plt.plot(df_conf["conf"], df_conf["detections"], marker="o")
plt.xlabel("Confidence threshold (conf)")
plt.ylabel("Number of detections")
plt.title("Detections vs confidence threshold")
plt.grid(True)
plt.show()
```

Listing 3: Sweep confidence threshold and count detections.

A.4 A.4 IoU Sweep (iou) and Grid Saving

```
conf_values = [0.1, 0.25, 0.5]
iou_values  = [0.3, 0.5, 0.7]

for conf in conf_values:
    for iou in iou_values:
        model.predict(
            source="zidane.jpg",
            conf=conf,
            iou=iou,
            save=True,
            project="runs/p1_conf_iou_grid",
            name=f"conf{conf}_iou{iou}"
        )
```

Listing 4: Run multiple (conf,iou) combinations and save outputs for comparison grids.

A.5 A.5 Class Filtering (People-only)

```
model = YOLO("yolov8n.pt")
names = model.names

# Find the class ID for "person"
person_id = None
for k, v in names.items():
    if v == "person":
        person_id = k
        break

print("person_id:", person_id)

# Predict keeping only persons
model.predict(
    source="bravo.jpeg",
    classes=[person_id],
    conf=0.25,
    save=True,
    project="runs/p1_people_only",
    name="bravo_people_only"
)
```

Listing 5: Filter detections by class ID (e.g., keep only 'person').

A.6 A.6 Dataset YAML (Vehicles) and Validation

```
model = YOLO("yolov8n.pt")
metrics = model.val(data="vehicles.yaml", split="val", imgsz=640)
print(metrics)
```

Listing 6: Validate pretrained YOLOv8n on vehicles dataset to measure domain shift.

A.7 A.7 Fine-tuning on Vehicles (Transfer Learning)

```
model = YOLO("yolov8n.pt")

model.train(
    data="vehicles.yaml",
    imgsz=640,
    epochs=80,
    batch=16,
    device=0,
    project="runs/vehicles2",
    name="yolov8n_ft_e80"
)
```

Listing 7: Fine-tune YOLOv8n on vehicles dataset and save runs.

A.8 A.8 Validate Fine-tuned Model

```
ft = YOLO("runs/vehicles2/yolov8n_ft_e80/weights/best.pt")
metrics_ft = ft.val(data="vehicles.yaml", split="val", imgsz=640)
print(metrics_ft)
```

Listing 8: Validate best checkpoint after fine-tuning and compare metrics.

A.9 A.9 Side-by-side Predictions (Baseline vs Fine-tuned)

```
baseline = YOLO("yolov8n.pt")
finetuned = YOLO("runs/vehicles2/yolov8n_ft_e80/weights/best.pt")

test_imgs = ["test1.jpg", "test2.jpg"]

baseline.predict(test_imgs, save=True, project="runs/vehicles2_compare",
    name="baseline")
finetuned.predict(test_imgs, save=True, project="runs/vehicles2_compare"
   , name="finetuned")
```

Listing 9: Run predict on the same test images for baseline vs fine-tuned outputs.

A.10 A.10 Video Detection + FPS Measurement

```
model = YOLO("yolov8n.pt")

start = time.time()
model.predict(
    source="test_video.mp4",
    conf=0.25,
    iou=0.7,
    vid_stride=1,
```

```
    save=True,
    project="runs/p3_video",
    name="detect"
)
elapsed = time.time() - start
print("Elapsed (s):", elapsed)

cap = cv2.VideoCapture("test_video.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
cap.release()
fps = n_frames / elapsed
print("Estimated FPS:", fps)
```

Listing 10: Video detection with runtime timing and FPS estimation.

A.11 A.11 Tracking with ByteTrack

```
model = YOLO("yolov8n.pt")
model.track(
    source="test_video.mp4",
    tracker="bytetrack.yaml",
    persist=True,
    conf=0.25,
    iou=0.7,
    save=True,
    project="runs/p3_video",
    name="track_bytetrack"
)
```

Listing 11: Tracking with ByteTrack (IDs + persist).

A.12 A.12 VisDrone Training

```
model = YOLO("yolov8n.pt")
model.train(
    data="VisDrone.yaml",
    imgsz=512,
    epochs=10,
    batch=16,
    device=0,
    project="runs/p4_visdrone",
    name="yolov8n_visdrone_e10"
)
```

Listing 12: Training YOLOv8 on VisDrone dataset.

A.13 A.13 Detection vs Segmentation on Video Frames

```
det = YOLO("yolov8n.pt")
seg = YOLO("yolov8n-seg.pt")

frames = ["frame1.png", "frame2.png"]  # extracted frames
det.predict(frames, save=True, project="runs/p5_detseg", name="det")
seg.predict(frames, save=True, project="runs/p5_detseg", name="seg")
```

Listing 13: Compare detection vs segmentation models on extracted frames.

A.14 A.14 Benchmark YOLOv8n/s/m on Video

```
models = {
    "yolov8n": YOLO("yolov8n.pt"),
    "yolov8s": YOLO("yolov8s.pt"),
    "yolov8m": YOLO("yolov8m.pt"),
}

for name, m in models.items():
    start = time.time()
    m.predict("test_video.mp4", conf=0.25, iou=0.7, vid_stride=1, save=
        False)
    elapsed = time.time() - start
    print(name, "elapsed:", elapsed)
```

Listing 14: Benchmark different model sizes (n/s/m) on the same video settings.

B Appendix B: Extra Figures

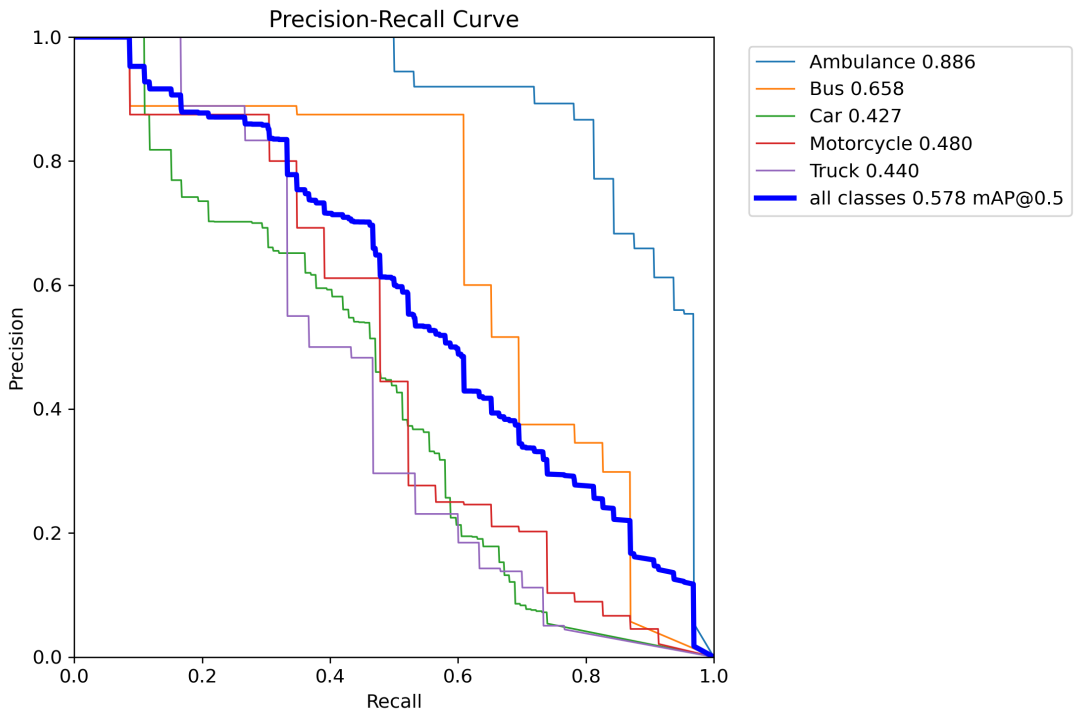


Figure 22: Vehicles: Precision-Recall curve

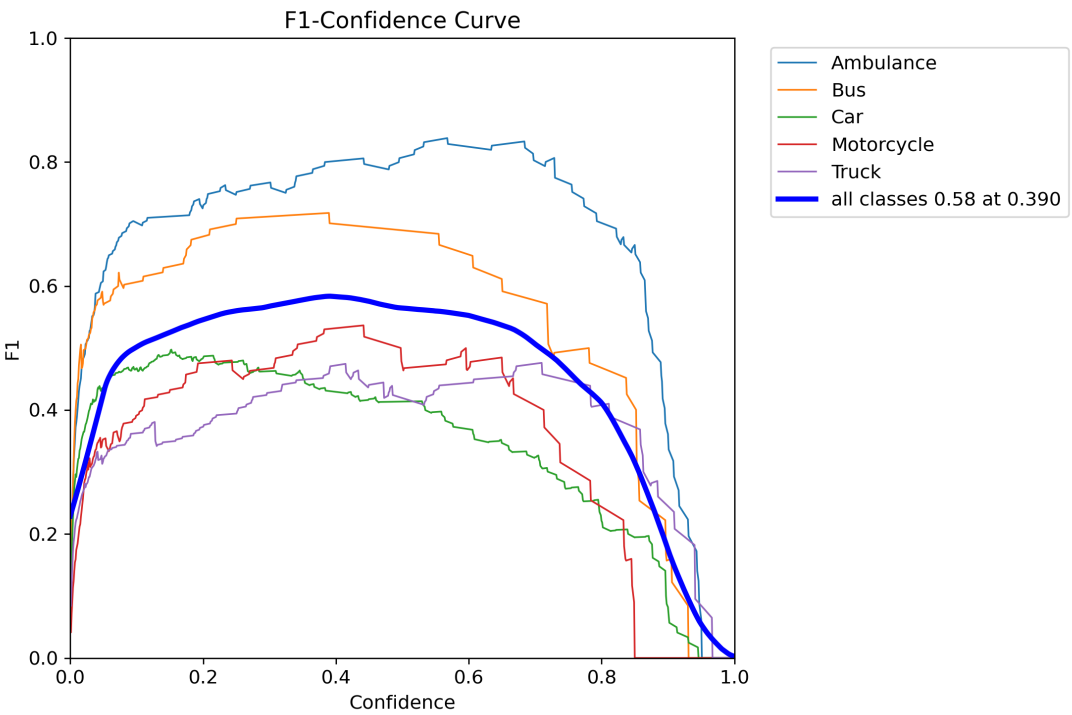


Figure 23: Vehicles: F1 curve vs confidence threshold